RAG ORIENTATION — STUDENT NOTES

From Documents to Intelligent AI Answers

Learning Objectives

After studying these notes, students should understand:

- Why normal LLMs may not answer questions about private documents.
- What RAG is and why we need it.
- Why Document Loaders, Text Splitting, Embeddings, Vector Stores and Retrievers are required.
- How these components work together.
- How to build a foundation for a complete RAG application.

---

1. WHY DO WE NEED RAG?

Consider a simple example.

DURGASOFT has a private PDF containing its course information.

A student asks an AI chatbot:

"What is the duration of the Python course?"

The information is available inside the private PDF.

But the LLM has not received that document.

Can the LLM automatically read the PDF from our computer?

No.

A private document stored on our computer is not automatically available to an LLM.

Our application must make the required information available.

Problems with Normal LLM Applications

- The LLM may not know our private information.
- Its existing knowledge may be outdated.
- It may generate incorrect information.
- It may confidently provide an answer without sufficient evidence.

Important Conclusions

- **Having information is different from making information available to AI.**
- **An LLM cannot automatically read our private documents.**
- **A powerful LLM can still give wrong answers when required information is missing.**
- **The right information is more important than simply having a powerful model.**

---

2. CAN WE DIRECTLY SUPPLY DOCUMENT TEXT TO AN LLM?

Yes.

Suppose our document contains:

Sample Course Information

- Course: Python Foundations
- Duration: 8 weeks
- Mode: Weekend Online
- Fee: Not mentioned

We can directly supply this small text to the LLM.

Then ask:

"What is the Python Foundations course duration?"

The LLM can answer:

"8 weeks."

But What About Large Documents?

Suppose our organisation has:

- 500-page PDFs
- 1,000 company documents
- 10,000 support articles

Sending all this information for every question is usually inefficient.

It may increase cost, slow down responses and exceed the model's context window.

A better solution is to find the relevant information first.

Important Conclusions

- **Small text can be supplied directly to an LLM.**
- **Every question does not require every document.**
- **Sending unnecessary information increases cost and processing.**
- **Find the required information first, then generate the answer.**

---

3. WHAT IS RAG?

RAG stands for Retrieval-Augmented Generation.

It is a technique used to generate answers by retrieving relevant external information and supplying it to an LLM as context.

Three Important Words

Retrieval: Finding relevant information.

Augmented: Adding useful information to the model's input.

Generation: Generating an answer using the LLM.

Normal LLM Flow

User Question → LLM → Answer

RAG Flow

User Question → Retrieve Relevant Information → LLM → Answer

Example

Student Question:

"What is the Python course duration?"

Retrieved Information:

"Python Foundations course duration is 8 weeks."

LLM Answer:

"According to the supplied course information, the Python Foundations course duration is 8 weeks."

Important Conclusions

- **RAG = Retrieval-Augmented Generation.**
- **Retrieval finds knowledge; generation creates the answer.**
- **RAG provides external information without retraining the LLM.**
- **RAG can reduce hallucinations but cannot guarantee correct answers.**
- **RAG is useful for private and domain-specific knowledge.**

---

4. COMPLETE RAG JOURNEY — MODULES 9 TO 13

Module	Topic	Main Responsibility
9	Document Loaders	Read external documents
10	Text Splitting	Divide large documents into chunks
11	Embeddings	Convert text into numerical vectors
12	Vector Stores	Store vectors and support similarity search
12	Retrievers	Find relevant document chunks
13	RAG	Generate answers using retrieved context

Complete RAG Architecture

Phase 1 — Prepare Knowledge

Documents

↓

Document Loaders

↓

Loaded Documents

↓

Text Splitting

↓

Text Chunks

↓

Embedding Model

↓

Numerical Vectors

↓

Vector Store

Phase 2 — Answer Questions

User Question

↓

Retriever (uses query embedding for vector search)

↓

Relevant Document Chunks

↓

Question + Retrieved Context

↓

LLM

↓

Final Answer

5. MODULE 9 — DOCUMENT LOADERS

What Is a Document Loader?

A **Document Loader** reads information from an external source and makes it available to our application in a useful document format.

Common Sources

- PDF files
- Text files
- CSV files
- Web pages
- Other supported document formats

Simple Flow

PDF File → Document Loader → LangChain Documents

LangChain Document

A LangChain Document mainly contains:

1. page_content

The actual document content.

2. metadata

Additional information about the document.

Example

page_content:

"Python Foundations course duration is 8 weeks."

metadata:

- source: courses.pdf
- page: 5

The available metadata fields depend on the loader.

Important Conclusions

- **Document Loaders bring external content into our application.**
- **page_content contains the main document content.**
- **metadata contains additional information about the document.**
- **Different file types may need different loaders.**
- **Document loading is the first step, not the complete RAG solution.**

---

6. MODULE 10 — TEXT SPLITTING

What Is Text Splitting?

Text Splitting is the process of dividing large text into smaller, manageable pieces called chunks.

Why Is Text Splitting Required?

Suppose a PDF contains 500 pages.

A student asks only about Python course duration.

We may not need all 500 pages.

We need the relevant part.

Simple Example

Large Course Document

↓

Chunk 1: Python Introduction

Chunk 2: Python Course Duration

Chunk 3: Python Syllabus

Chunk 4: Java Introduction

Chunk 5: Java Course Duration

Important Terms

Chunk: A smaller piece of document content.

Chunk Size: The configured size limit for a chunk.

Chunk Overlap: Some content repeated between neighbouring chunks to help preserve context.

Important Conclusions

- **Text splitting divides large content into smaller chunks.**
- **Every question does not require the complete document.**
- **Good chunks improve the chances of useful retrieval.**
- **Very small chunks may lose context.**
- **Very large chunks may include unnecessary information.**

---

7. MODULE 11 — EMBEDDINGS

What Is an Embedding?

An **embedding** is a numerical representation of text or other supported data generated by an embedding model.

Why Do We Need Embeddings?

Consider these two sentences:

1. "How can I learn Python?"
2. "What is the best way to study Python programming?"

The words are not exactly the same.

But the meanings are similar.

Embedding models help represent this similarity numerically.

Simple Flow

Text

↓

Embedding Model

↓

Numerical Vector

Example

"Learn Python"

↓

Embedding Model

↓

[0.12, -0.45, 0.83, ...]

These values are only for illustration.

Actual values depend on the embedding model.

Important Conclusions

- **Embeddings represent meaning using numbers.**
- **Different sentences can have similar meanings.**
- **Embeddings help us search by meaning, not just matching words.**
- **Embedding Models and Chat Models perform different tasks.**

- **Embeddings help in retrieval; they do not generate final answers.**

---

8. MODULE 12 — VECTOR STORES

What Is a Vector Store?

A **Vector Store** stores vector embeddings and supports similarity searching.

Why Do We Need Vector Stores?

Suppose our application generates embeddings for 10,000 document chunks.

We need a way to organise and search them efficiently.

Simple Flow

Text Chunks

↓

Embedding Model

↓

Embeddings

↓

Vector Store

Common Technologies

FAISS: Used for efficient vector similarity search.

Chroma: A vector database for storing and searching embeddings and related document information.

Important Conclusions

- **Generating embeddings and storing embeddings are different tasks.**
- **Vector Stores support efficient similarity searching.**
- **Vector Stores help locate semantically related content.**
- **A Vector Store is not an LLM.**
- **Storing knowledge is different from generating an answer.**

---

9. MODULE 12 — RETRIEVERS

What Is a Retriever?

A **Retriever** is a component that receives a query and returns relevant documents.

Simple Flow

Student Question

↓

Retriever

↓

Search Relevant Information

↓

Relevant Document Chunks

Example

Question:

"What is the Python course duration?"

Retrieved Chunk:

"Python Foundations course duration is 8 weeks."

The Retriever finds the information.

It does not generate the final answer.

Vector Store vs Retriever

Vector Store	**Retriever**
Stores and searches vectors	Retrieves relevant documents
Supports similarity searching	Provides a query-to-documents interface
Manages vector search operations	May use a Vector Store internally

Important Conclusions

- **Retrievers find relevant information.**
- **Retrievers do not generate the final answer.**
- **Vector Stores and Retrievers are different components.**

- **Good retrieval improves the chances of a good answer.**
- **Wrong retrieval can produce poor answers, even with a powerful LLM.**

---

10. MODULE 13 — COMPLETE RAG APPLICATION

A complete basic RAG application combines all the previous components.

It has two main phases.

Phase 1 — Ingestion Pipeline

Purpose: Prepare and store document knowledge.

Documents → Load → Split → Embed → Store

We normally perform this when preparing the document collection and repeat the required steps when documents change.

Phase 2 — Retrieval and Generation Pipeline

Purpose: Answer user questions.

Question → Retrieve Relevant Chunks → Build Prompt → LLM → Answer

For vector-based retrieval, the question is also represented as an embedding.

Complete Example

Our sample PDF contains:

"Python Foundations course duration is 8 weeks."

Student asks:

"What is the Python course duration?"

The Retriever finds the relevant chunk.

The application supplies this chunk along with the question.

The LLM generates the answer:

"The Python Foundations course duration is 8 weeks."

Important Conclusions

- **Ingestion prepares knowledge for searching.**
- **Retrieval finds the useful information for a question.**
- **Generation creates the answer using the retrieved context.**
- **Basic RAG does not require retraining the model.**
- **When required information is unavailable, the application should avoid guessing.**

---

11. RAG AND PREVIOUS LANGCHAIN MODULES

Our earlier LangChain concepts are also useful for building RAG applications.

Previous Module	Use in RAG
Module 4 — Messages	Communicate with the LLM
Module 5 — Prompt Templates	Combine question and context
Module 6 — Output Parsers	Process generated answers
Module 7 — LCEL	Connect retrieval and generation components
Module 8 — Conversation History	Support follow-up questions

Conversational RAG Example

User: "What is the Python course duration?"

AI: "8 weeks."

User: "What about its syllabus?"

The application can use previous conversation history to understand that "its" refers to the Python course.

Important Conclusions

- **RAG builds on the LangChain concepts we already learned.**
- **Prompts help the LLM use retrieved information.**
- **LCEL can connect RAG components into workflows.**
- **Conversation History helps understand follow-up questions.**

---

12. IMPORTANT DIFFERENCES

Concept	Main Responsibility
Document Loader	Read external content
Text Splitter	Divide content
Embedding Model	Create numerical representations
Vector Store	Store and search vectors
Retriever	Return relevant documents
LLM	Generate answers
RAG	Combine retrieval with generation

Common Misunderstandings

1. Does every RAG application need a Vector Store?

No. Other retrieval methods can also be used.

2. Does every document require Text Splitting?

No. Small documents may not require further splitting.

3. Does RAG retrain the LLM?

No. Basic RAG supplies external context without changing model weights.

4. Can RAG guarantee correct answers?

No. Retrieval or generation may still produce mistakes.

5. Does a Retriever generate the answer?

No. Its responsibility is to find relevant documents.

---

13. FIVE PRACTICE MCQs

MCQ 1

What is the main responsibility of a Document Loader?

A. Generate embeddings

B. Read external document content

C. Generate the final answer

D. Train the model

Correct Answer: B

Explanation: Document Loaders bring external content into the application.

Learning Conclusion: Before AI can use external knowledge, the application must access it.

MCQ 2

Why do we use Text Splitting?

A. To train an LLM

B. To create manageable chunks

C. To generate API keys

D. To delete documents

Correct Answer: B

Explanation: Text Splitting divides large documents into smaller pieces.

Learning Conclusion: The right piece of information can be more useful than the entire document.

MCQ 3

What does an Embedding Model generate?

A. A PDF file

B. A numerical vector

C. The final natural-language answer

D. A Python program

Correct Answer: B

Explanation: Embedding Models produce numerical representations.

Learning Conclusion: Embeddings help represent meaning in a mathematical form.

MCQ 4

What is the main job of a Retriever?

A. Store passwords

B. Generate final answers

C. Find relevant documents

D. Train a new LLM

Correct Answer: C

Explanation: Retrievers return relevant documents for a query.

Learning Conclusion: Retrieval finds the knowledge; generation explains it.

MCQ 5

What is the main idea of RAG?

A. Retrain the LLM with every document

B. Retrieve useful information and generate an answer using it

C. Replace the LLM completely

D. Convert Python into another language

Correct Answer: B

Explanation: RAG combines external information retrieval with LLM-based generation.

Learning Conclusion: Find the right knowledge before generating the answer.

---

14. TEN MASTER CONCLUSIONS — MUST REMEMBER

1. **AI cannot automatically use private documents stored on our computer.**
2. **The right information is more important than simply having a powerful model.**
3. **Document Loaders bring knowledge into our application.**
4. **Text Splitting divides large knowledge into smaller, useful pieces.**

5. **Embeddings help represent meaning using numbers.**
6. **Vector Stores help store and search embeddings efficiently.**
7. **Retrievers find information; LLMs generate answers.**
8. **RAG does not mean retraining the LLM.**
9. **The quality of retrieved information directly affects answer quality.**
10. **The real objective of RAG is to generate answers supported by relevant information.**

---

15. FINAL REVISION — ONE-LINE MEMORY TRICK

LOAD → SPLIT → EMBED → STORE → RETRIEVE → GENERATE

- **LOAD:** Bring document content into our application.
- **SPLIT:** Divide large content into smaller chunks.
- **EMBED:** Convert chunks into numerical vectors.
- **STORE:** Organise embeddings for searching.
- **RETRIEVE:** Find relevant chunks for a question.
- **GENERATE:** Use the relevant context to produce an answer.

Final Learning Message

"A powerful AI model becomes much more useful when we give it the right information at the right time."

NEXT MODULE — MODULE 9: DOCUMENT LOADERS

We will learn:

- What is a LangChain Document?
- What is page_content?
- What is metadata?
- How do Document Loaders work?
- How can we load PDF and text files?
- How can we inspect the loaded information?

Our RAG journey starts with Document Loaders!

AI with Durga Sir | LangChain Student Notes